Assignment 1

1. A 5-meter ladder is initially standing upright against a wall. Its bottom end is at (0,0), and its top end is at (0,5).

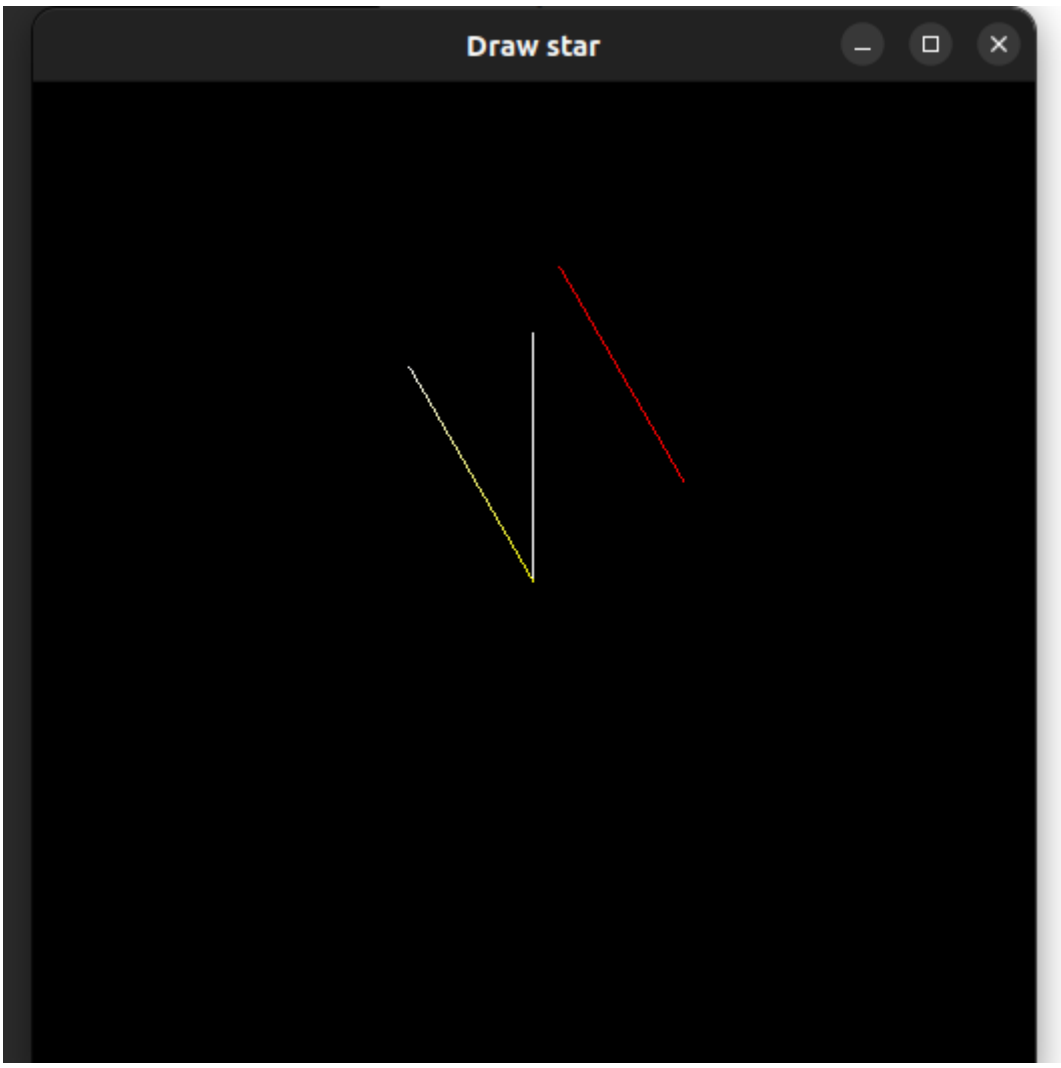
a. Now, the ladder undergoes the following transformations:. It is tilted 30◦ counterclockwise around its bottom end (0,0).

b. Then, the entire ladder is moved (translated) so that its bottom end is now at (3,2).

Apply necessary transformations and obtain the desired final figure.

```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(5);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(-10,10,-10,10);
}

void operation(int x0,int y0,int x1,int y1,int degree){

	float x11,y11,x22,y22;
	float radians = degree * M_PI / 180.0;

	x11=x1*cos(radians)-y1*sin(radians);
	y11=y1*cos(radians)+x1*sin(radians);
	glVertex2f(x11,y11);

	glColor3f(1,1,0);
	x22=x0*cos(radians)-y0*sin(radians);
	y22=y0*cos(radians)+x0*sin(radians);
	glVertex2f(x22,y22);

	glColor3f(1,0,0);
	glVertex2f(x11+3,y11+2);
	glVertex2f(x22+3,y22+2);

}

void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glBegin(GL_LINES);
```

```
	int x0=0,y0=0,x1=0,y1=5,degree=30;

	glVertex2f(x0,y0);
	glVertex2f(x1,y1);

	operation(x0,y0,x1,y1,degree);

	glEnd();

	glFlush();
}



int main(int argc,char** argv) {

	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
	glutInitWindowSize(500,500);
	glutInitWindowPosition(1200,550);
	glutCreateWindow("Draw star");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```

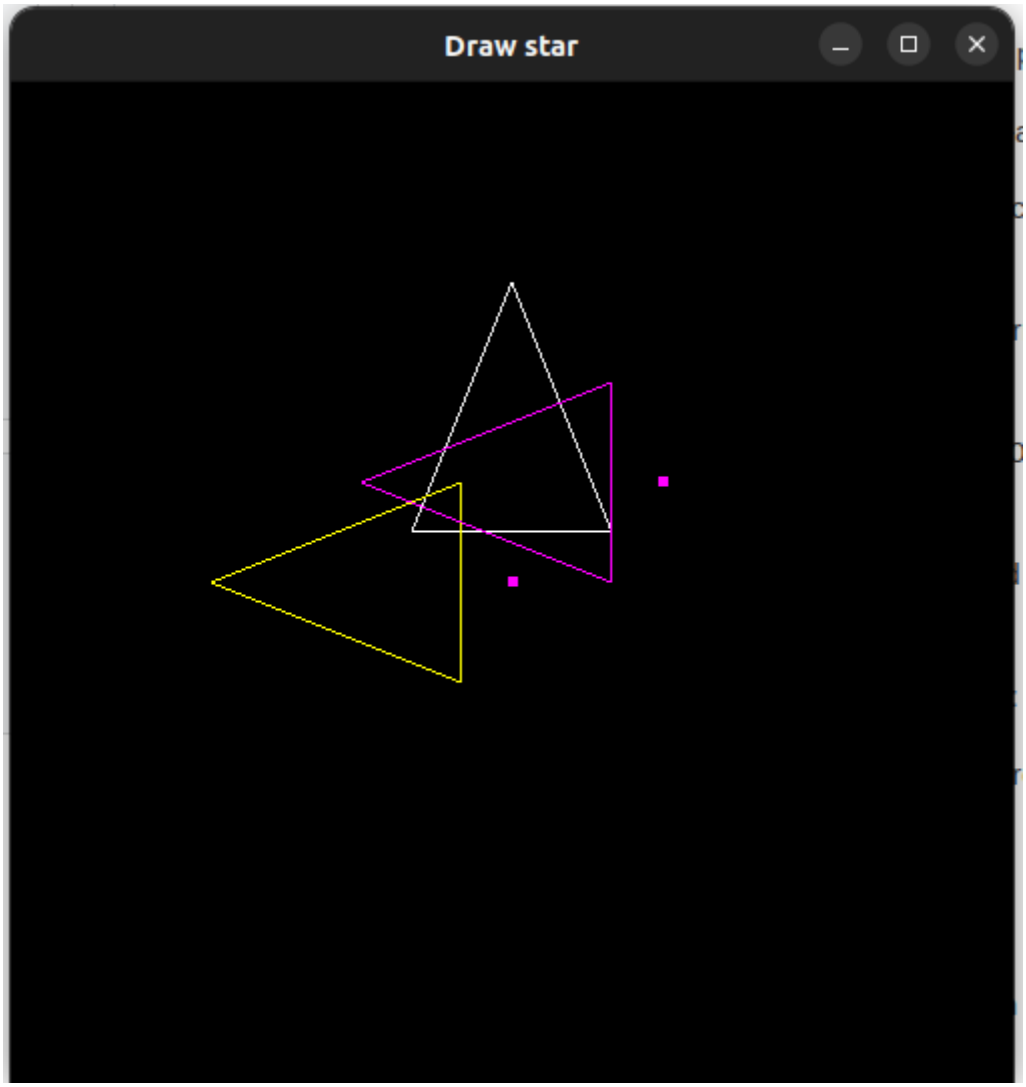


2. Create and rotate a triangle about the origin and a fixed point.

```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(5);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(-10,10,-10,10);
}

void rotate(int x0,int y0,int x1,int y1,int x2,int y2,int degree){

	float x11,y11,x22,y22,x00,y00;
	float radians = degree * M_PI / 180.0;

	x00=x0*cos(radians)-y0*sin(radians);
	y00=y0*cos(radians)+x0*sin(radians);
```

```
        x11=x1*cos(radians)-y1*sin(radians);
        y11=y1*cos(radians)+x1*sin(radians);

        x22=x2*cos(radians)-y2*sin(radians);
        y22=y2*cos(radians)+x2*sin(radians);

        glColor3f(1,1,0);

        glVertex2f(x00,y00);
        glVertex2f(x11,y11);

        glVertex2f(x11,y11);
        glVertex2f(x22,y22);

        glVertex2f(x22,y22);
        glVertex2f(x00,y00);

        glColor3f(1,0,1);

        glVertex2f(x00+3,y00+2);
        glVertex2f(x11+3,y11+2);

        glVertex2f(x11+3,y11+2);
        glVertex2f(x22+3,y22+2);

        glVertex2f(x22+3,y22+2);
        glVertex2f(x00+3,y00+2);

}


void display() {
        glClear(GL_COLOR_BUFFER_BIT);
        glBegin(GL_LINES);

        int x0=-2,y0=1,x1=0,y1=6,x2=2,y2=1;

        glVertex2f(x0,y0);
        glVertex2f(x1,y1);

        glVertex2f(x1,y1);
        glVertex2f(x2,y2);

        glVertex2f(x2,y2);
        glVertex2f(x0,y0);

        rotate(x0,y0,x1,y1,x2,y2,90);
        rotate(x0,y0,x1,y1,x2,y2,90);

        glEnd();
        glBegin(GL_POINTS);
```

```
	glVertex2f(0,0);
	glVertex2f(3,2);
	glEnd();

	glFlush();
}



int main(int argc,char** argv) {

	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
	glutInitWindowSize(500,500);
	glutInitWindowPosition(1200,550);
	glutCreateWindow("Draw star");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```

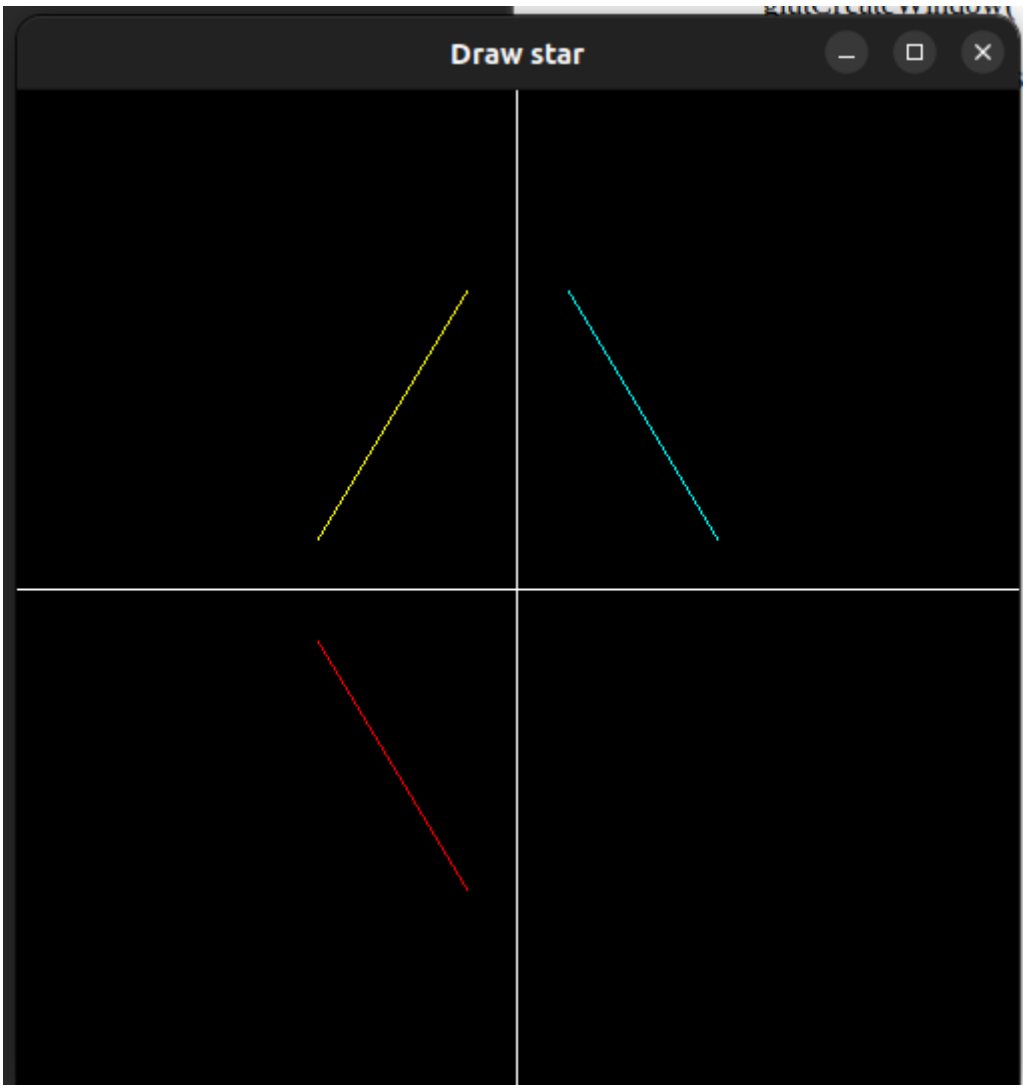


3. Reflect a line first about the x-axis and then about the y-axis.

```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(5);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(-10,10,-10,10);
}

void reflect(int x0,int y0,int x1,int y1){


	glVertex2f(x0,y0);
	glVertex2f(x1,y1);
}
```

```
void display() {
	glClear(GL_COLOR_BUFFER_BIT);
	glBegin(GL_LINES);

	int x0=-4,y0=1,x1=-1,y1=6;

	glColor3f(1,1,1);

	glVertex2f(0,10);
	glVertex2f(0,-10);
	glVertex2f(10,0);
	glVertex2f(-10,0);

	glColor3f(1,1,0);
	glVertex2f(x0,y0);
	glVertex2f(x1,y1);

	glColor3f(1,0,0);
	reflect(x0,-y0,x1,-y1);

	glColor3f(0,1,1);
	reflect(-x0,y0,-x1,y1);

	glEnd();

	glFlush();
}



int main(int argc,char** argv) {

	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
	glutInitWindowSize(500,500);
	glutInitWindowPosition(1200,550);
	glutCreateWindow("Draw star");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```

4. You are given the



following geometric shapes in a local coordinate system:

a. A rectangle (representing the house) with bottom-left corner at (0,0) and top-right corner at (6,9).

b. A triangle (representing the roof) with vertices at: A=(0,0), B=(2,0), C=(1,1) i. You need to:

c. Translate the rectangle so that its left edge is positioned at x=5 on the x-axis.

d. Place the triangle on top of the rectangle, ensuring that:

1. The base of the triangle aligns with the top edge of the rectangle.

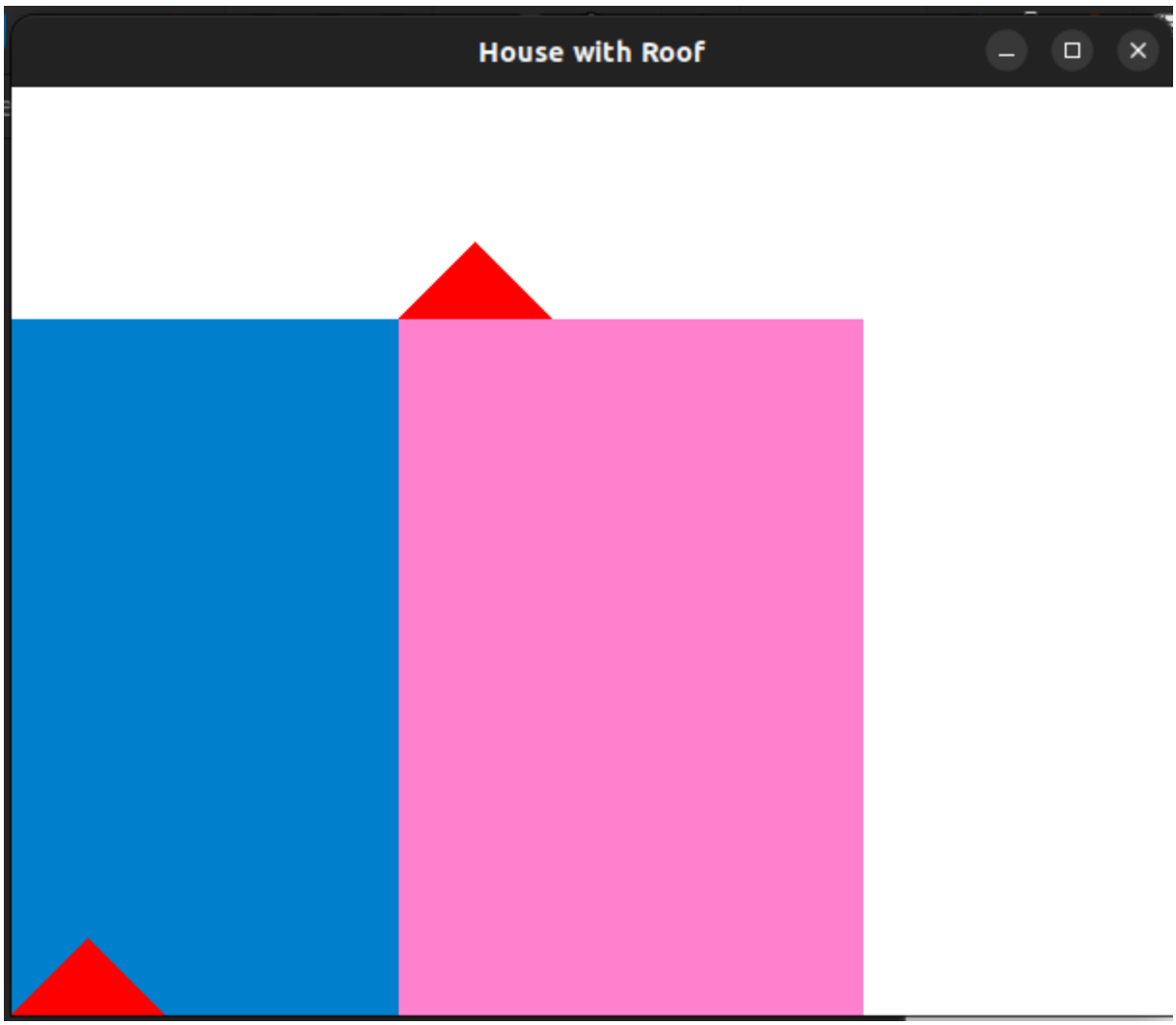
ii. The triangle remains centered over the rectangle.

' Apply the transformations and show the final figure

```
#include <GL/glut.h>
float x1 = 0,Y = 0;
float x2 = 0,y2 = 9;
float x3 = 6,y3 = 9;
```

```
float x4 = 6,y4 = 0;
float transX = 5;
void translate(){
        glColor3f(2.0, 0.5, 0.8);
        glBegin(GL_QUADS);
                glVertex2f(x1+transX,Y);
                glVertex2f(x2+transX,y2);
                glVertex2f(x3+transX,y3);
                glVertex2f(x4+transX,y4);
        glEnd();
        glFlush();
        glColor3f(3.0,0.0,0.0);


        glBegin(GL_TRIANGLES);
        glVertex2f(0.0f+transX,0.0f+9.0f);
        glVertex2f(2.0f+transX,0.0f+9.0f);
        glVertex2f(1.0f+transX,1.0f+9.0f);
        glEnd();

   glFlush();
}
void display(){
   glClear(GL_COLOR_BUFFER_BIT);
   glColor3f(0.0, 0.5, 0.8);
   glBegin(GL_QUADS);
      glVertex2f(0.0f,0.0f);
      glVertex2f(0.0f,9.0f);
      glVertex2f(6.0f,9.0f);
      glVertex2f(6.0f,0.0f);
   glEnd();

   glColor3f(1.0,0.0,0.0);


   glBegin(GL_TRIANGLES);
      glVertex2f(0.0f,0.0f);
      glVertex2f(2.0f,0.0f);
      glVertex2f(1.0f,1.0f);
   glEnd();

   glFlush();
   translate();
}

void init(){
   glClearColor(1,1,1,1);
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(0,15,0,12);
   glMatrixMode(GL_MODELVIEW);
}
```

```
int main(int argc,char** argv){
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600,480);
    glutInitWindowPosition(100,100);
    glutCreateWindow("House with Roof");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

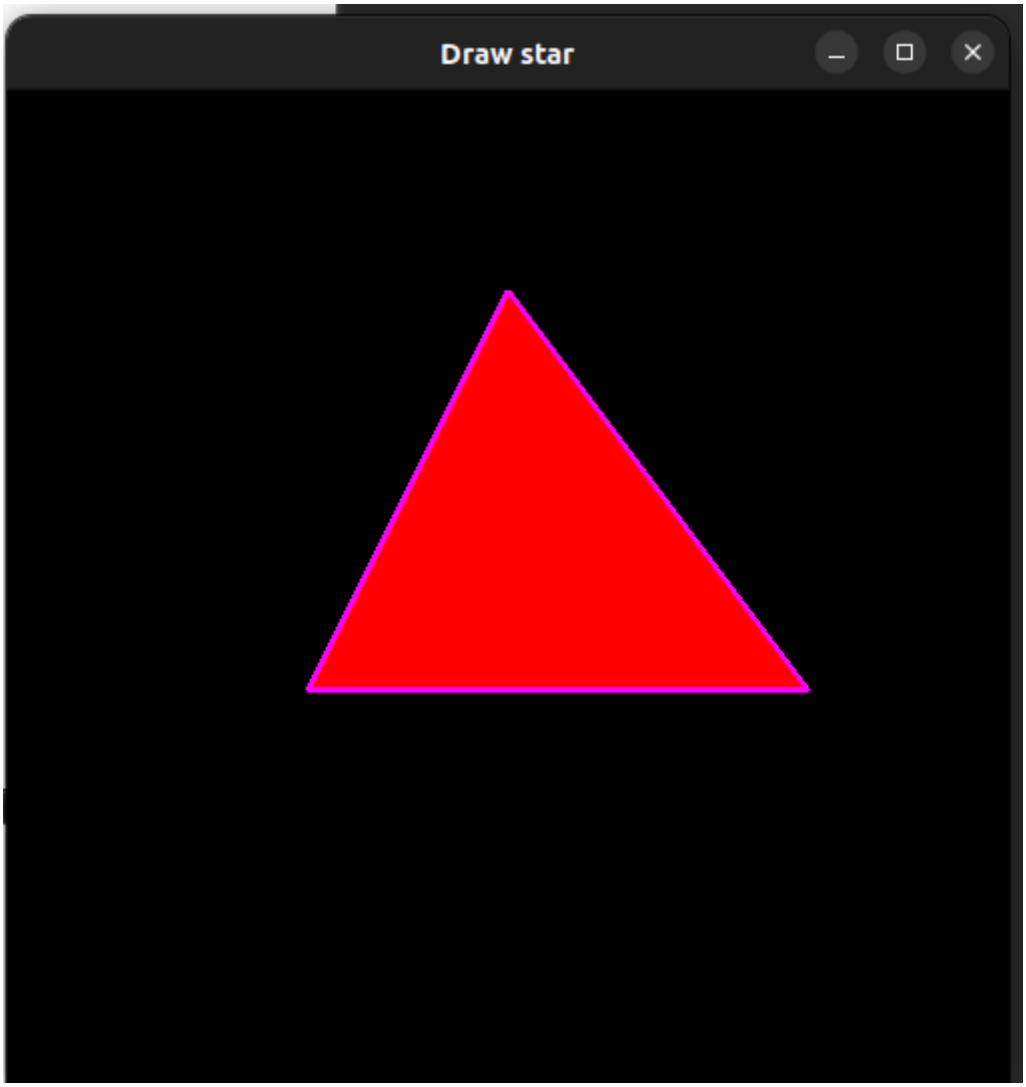

House with Roof

5. Write a program to implement Boundary Fill Algorithm using recursion (8-connected)

```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(1);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(0,500,0,500);
}

void boundary(int x,int y,vector<float>&b,vector<float>&f){

	float current[3];
	glReadPixels(x,y,1,1,GL_RGB,GL_FLOAT,current);

	if((current[0]!=b[0]||current[1]!=b[1]||current[2]!=b[2]) &&
	(current[0]!=f[0]||current[1]!=f[1]||current[2]!=f[2])){

		glBegin(GL_POINTS);
		glColor3f(1,0,0);
		glVertex2i(x,y);
		glEnd();
		glFlush();

		boundary(x,y+1,b,f);
		boundary(x,y-1,b,f);
		boundary(x+1,y+1,b,f);
		boundary(x+1,y-1,b,f);
		boundary(x-1,y+1,b,f);
		boundary(x-1,y-1,b,f);
		boundary(x+1,y,b,f);
		boundary(x-1,y,b,f);

	}
}


void display() {

	glClear(GL_COLOR_BUFFER_BIT);
	glLineWidth(3.0);
	glBegin(GL_LINE_LOOP);

	vector<float>b={1,0,1},f={1,0,0};
	glColor3f(1,0,1);
```

```
	glVertex2f(250,400);
	glVertex2f(150,200);
	glVertex2f(400,200);

	glEnd();
	glFlush();
	//glVertex2f(250,250);
	boundary(250,250,b,f);
}



int main(int argc,char** argv) {

	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
	glutInitWindowSize(500,500);
	glutInitWindowPosition(1200,550);
	glutCreateWindow("Draw star");
	init();
	glutDisplayFunc(display);
	glutMainLoop();
	return 0;
}
```

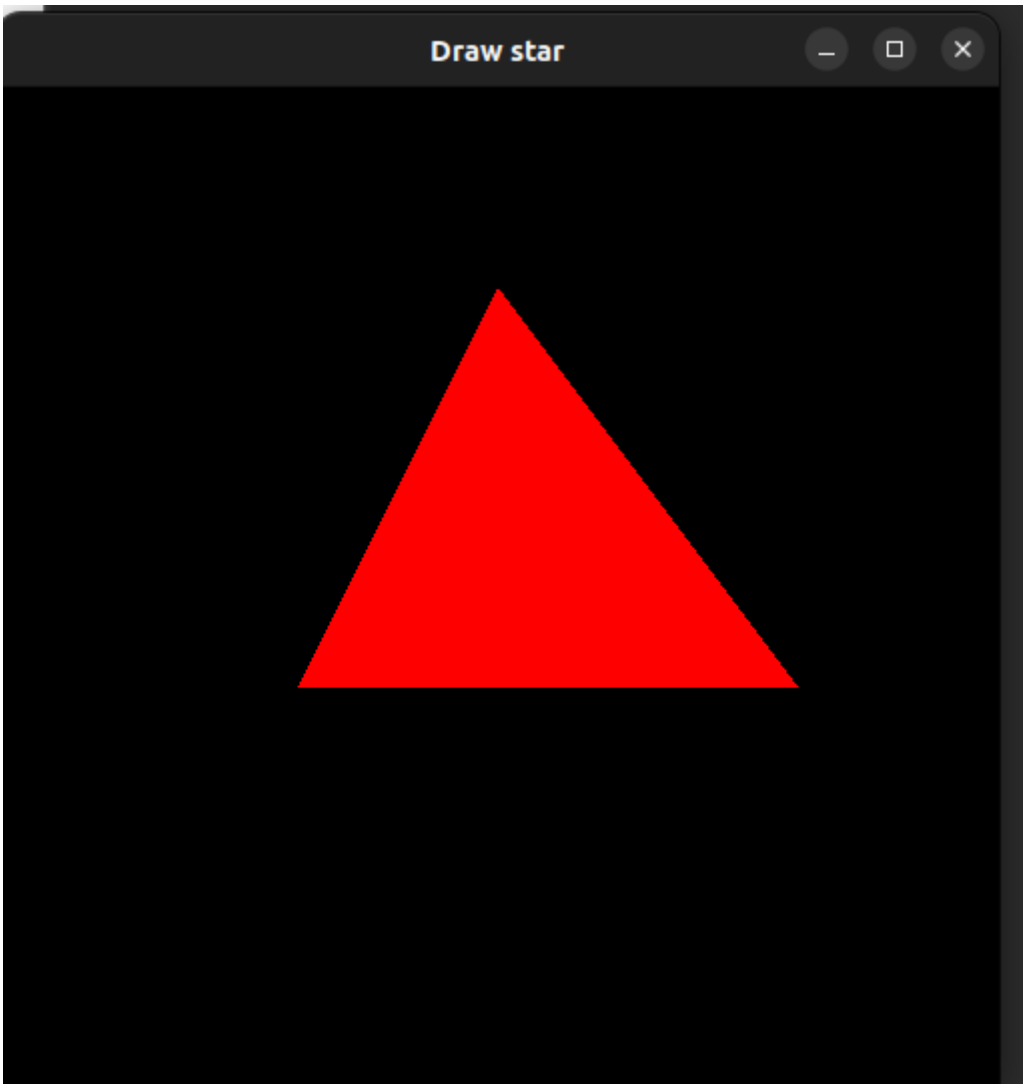
6. Write a program for Flood Fill Algorithm




```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(1);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluOrtho2D(0,500,0,500);
}

void flood(int x,int y,vector<float>&n,vector<float>&o){

	float current[3];
	glReadPixels(x,y,1,1,GL_RGB,GL_FLOAT,current);

	if((current[0]==o[0]&&current[1]==o[1]&&current[2]==o[2])){
```

```
                glBegin(GL_POINTS);
                glColor3f(1,0,0);
                glVertex2i(x,y);
                glEnd();
                glFlush();

                flood(x,y+1,n,o);
                flood(x,y-1,n,o);
                flood(x+1,y+1,n,o);
                flood(x+1,y-1,n,o);
                flood(x-1,y+1,n,o);
                flood(x-1,y-1,n,o);
                flood(x+1,y,n,o);
                flood(x-1,y,n,o);

        }
}


void display() {

        glClear(GL_COLOR_BUFFER_BIT);
        glLineWidth(3.0);
        glBegin(GL_POLYGON);

        vector<float>n={1,0,0},o={1,1,1};
        glColor3f(1,1,1);

        glVertex2f(250,400);
        glVertex2f(150,200);
        glVertex2f(400,200);

        glEnd();
        glFlush();
        //glVertex2f(250,250);
        flood(250,250,n,o);
}



int main(int argc,char** argv) {

        glutInit(&argc,argv);
        glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
        glutInitWindowSize(500,500);
        glutInitWindowPosition(1200,550);
        glutCreateWindow("Draw star");
        init();
        glutDisplayFunc(display);
        glutMainLoop();
        return 0;
```

}



7. Write a program to:

⑩ Draw a polygon

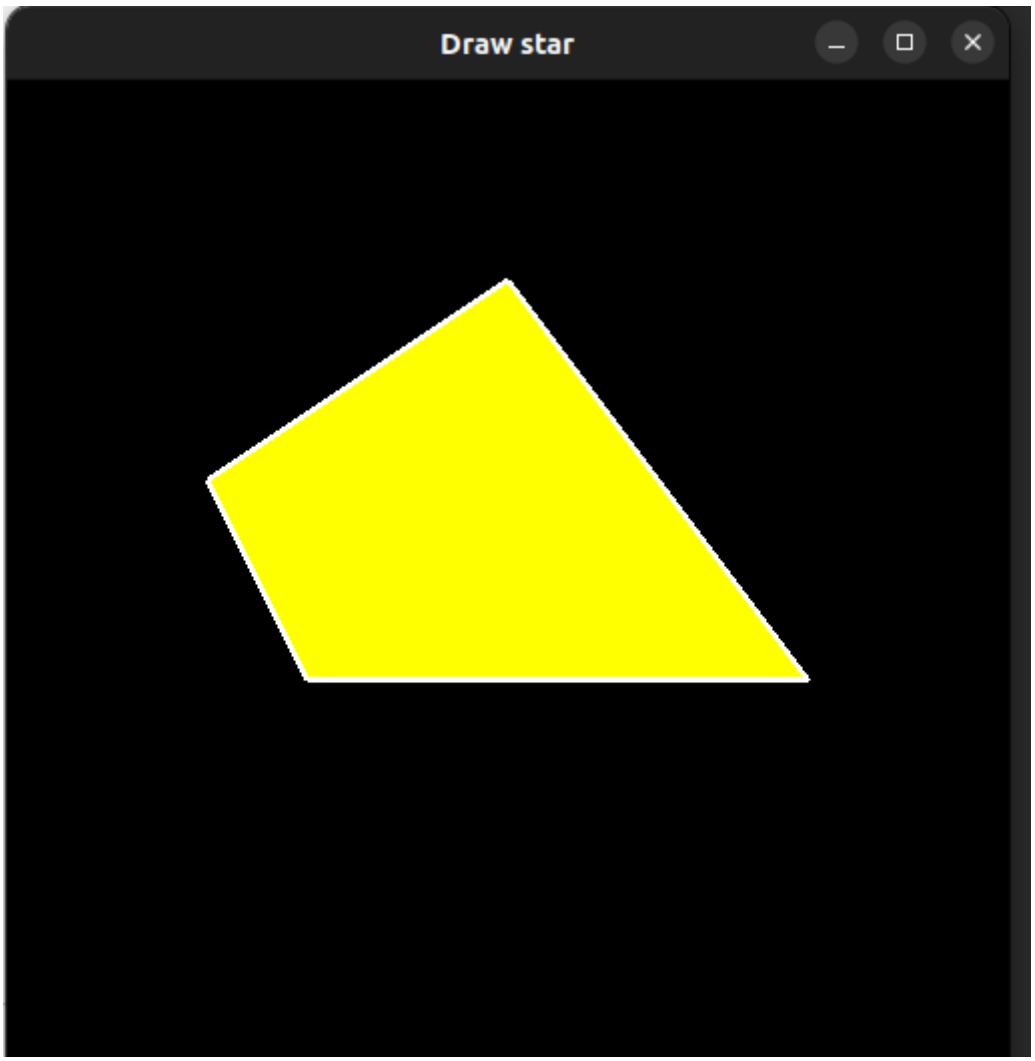
⑩ Fill it using Boundary Fill

⑩ Allow user to click seed point using mouse

```
#include <GL/glut.h>
#include<bits/stdc++.h>
using namespace std;

vector<float>b={1,1,1},f={1,1,0};
void init() {
	glClearColor(0,0,0,1);
	glColor3f(1,1,1);
	glPointSize(1);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
```

```
        gluOrtho2D(0,500,0,500);
}

void flood(int x,int y,vector<float>&b,vector<float>&f){

        float current[3];
        glReadPixels(x,y,1,1,GL_RGB,GL_FLOAT,current);

        if((current[0]!=b[0]||current[1]!=b[1]||current[2]!=b[2]) &&
        (current[0]!=f[0]||current[1]!=f[1]||current[2]!=f[2])){

                glBegin(GL_POINTS);
                glColor3f(1,1,0);
                glVertex2i(x,y);
                glEnd();

                flood(x,y+1,b,f);
                flood(x,y-1,b,f);
                flood(x+1,y+1,b,f);
                flood(x+1,y-1,b,f);
                flood(x-1,y+1,b,f);
                flood(x-1,y-1,b,f);
                flood(x+1,y,b,f);
                flood(x-1,y,b,f);

        }
}


void display() {

        glClear(GL_COLOR_BUFFER_BIT);
        glLineWidth(3.0);
        glBegin(GL_LINE_LOOP);

        glColor3f(1,1,1);

        glVertex2f(250,400);
        glVertex2f(100,300);
        glVertex2f(150,200);
        glVertex2f(400,200);

        glEnd();
        glFlush();
        //glVertex2f(250,250);

        //boundary(250,250,b,f);
}

void mouse(int button, int state, int x, int y) {
   if(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
      int seedX = x;
```

```
        int seedY =500 - y; // Flip y coordinate
        flood(seedX, seedY, b,f);
        glFlush();
    }
}

int main(int argc,char** argv) {

            glutInit(&argc,argv);
            glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
            glutInitWindowSize(500,500);
            glutInitWindowPosition(1200,550);
            glutCreateWindow("Draw star");
            init();
            glutDisplayFunc(display);
            glutMouseFunc(mouse);
            glutMainLoop();
            return 0;
}
```


Draw star